

Architecture & Trade-offs

In-Memory Account Ledger Core

This document focuses on the production questions raised by the implementation. It does not repeat the supplied event stream or restate the individual business rules from Part 1. The current implementation is deliberately small: an append-only in-memory ledger, a replay engine, a console runner, and tests. The decisions below describe what would change when moving from this exercise to a real banking system.

1. Append-only at scale

The append-only approach is valuable because it gives a clear audit trail. A correction is represented by a new compensating record rather than by changing an old record. That makes the history explainable and allows a balance to be rebuilt from the entries.

At 100x the current volume, the first problem would probably be replay and query cost, not the append-only rule itself. In the exercise, entries, authorizations, fees, accruals, rejected events, and reports are all held in process. These collections grow for the complete run and are scanned repeatedly when balances and historical days are calculated. Memory usage and repeated scans would grow with the event count. A full replay also becomes slower, especially when a late value-dated entry causes several historical days to be checked again.

The cheapest structural change I would make first is to move the immutable event and entry log to durable storage while keeping the domain/replay model. I would add indexes by account and value date, and by account and booking sequence. I would also add periodic snapshots or materialized balance projections. A snapshot would allow replay to start from the latest trusted point instead of replaying the account complete history every time. The source log remains authoritative; the snapshot is only a performance optimization and can be rebuilt.

I would not begin by sharding or introducing many services. Those changes increase operational complexity before the simpler bottleneck is measured. The initial scalable shape would be: append-only durable journal, indexed projections, background projection workers, and a reconciliation job that compares projections with a replay of the source journal.

What remains unbounded

- The source event and ledger-entry history grows continuously.
- In-memory replay collections grow for the lifetime of the process.
- Audit and rejected-event records grow unless retention and archival rules are defined.
- Snapshots and projections need a lifecycle, but they must never replace the authoritative journal.

Trade-off

Compaction reduces read cost but can damage auditability if it deletes the original history. I would therefore archive old immutable records rather than delete them, and retain a verifiable chain or reconciliation information for the archived period.

2. Value-dated entries in production

Value dating is useful because an event may be received today but economically effective on an earlier or later date. It also creates a larger operational and regulatory surface than a simple posting date. A balance depends on two questions: when the entry became known to the system and which date it affects.

In a UAE-licensed bank, I would expect this to affect customer statements, interest calculation, overdraft decisions, fees, cut-off times, settlement and reconciliation, exception handling, and audit evidence. Operations would need to distinguish the original received time from the effective value date and preserve the reason, source, user or system actor, and approval trail for a backdated change.

It also creates customer-impact risk. A backdated debit can change a previously reported balance, available funds, interest, or fee eligibility. If that change is allowed after a statement or reporting period has closed, the system must show whether the result is a correction, an adjustment, or a new event. It should not silently rewrite an old statement.

The regulatory surface includes record retention, traceability, controlled access, reconciliation, suspicious or unusual activity review, and the ability to produce a complete explanation of a customer balance. The exact obligations would be confirmed with the bank compliance and legal teams; I would not assume that a technical value date alone satisfies them.

Control before go-live

I would add a controlled backdating workflow. A value date earlier than the current business date would require a reason code, a permitted-role approval, a limit check, and an immutable audit record containing received time, effective time, old and new projected balances, and downstream impacts. The system would also run reconciliation and customer-statement impact checks before posting. This is the first control I would require before production.

3. Authorization lifecycle

The current model has approved, declined, and settled states. A real authorization needs more terminal paths because a hold can end without a matching settlement. I would make these transitions explicit and append an event for each one; I would not simply change the hold row without history.

End state	Real-world scenario	Mandated system behavior
Declined	The requested amount would exceed available funds, the account is blocked, or a risk rule rejects the request.	Record the authorization decision and reason. Do not create an active hold or change the ledger balance. Return a stable decision to the caller.
Expired	The merchant never completes the purchase within the allowed authorization window.	Append an expiry event, release the remaining hold, retain the original authorization and expiry reason, and make the release visible in audit/reporting.
Cancelled / voided	The merchant or payment channel cancels the purchase before settlement.	Require an authorized cancellation request, append a cancellation event, release the hold, and prevent later settlement unless a new authorization is created.
Reversed	A previously approved or settled payment is cancelled because of a duplicate, failed completion, customer refund, or operational correction.	Append a compensating event linked to the original authorization/settlement. Never delete the original record; require reason and actor/source details.
Partially settled / released	The final amount is lower than the hold, or only part of an order is fulfilled.	Post the actual settled amount, reduce or release the remaining hold, and close the authorization when no amount remains. Keep the full settlement chain.
Rejected / invalidated	A duplicate, malformed, unknown, replayed, or fraud-screened request is received.	Record the rejection and reason, make no financial movement, and enforce idempotency so retries cannot create a second hold.
System/manual recovery	A timeout or external-network failure leaves the result uncertain.	Place the authorization in an explicit pending/reconciliation state. Do not guess success or release funds without a controlled recovery decision.

The important design point is that hold lifecycle and ledger movement are related but not identical. Releasing a hold changes available balance; only a valid financial posting changes ledger balance.

4. What I cut and why

The cuts were intentional and made to stay within the stated assessment boundary. Each one defers a real production risk rather than solving it.

Simplification	Why it was cut	Production risk deferred
No web/API layer	The requirement asks for a console runner and test suite, not transport.	Authentication, authorization, request validation, rate limiting, API versioning, and client error contracts are not addressed.
No database or persistence	The exercise explicitly requires in-memory behavior.	Data is lost on process exit; there is no durable audit trail, recovery, transaction isolation, or operational backup.
Plain in-memory collections	They are enough to demonstrate replay and immutable records.	Memory growth, slow historical scans, process-local state, and no multi-instance consistency.

Simplification	Why it was cut	Production risk deferred
Fixed event stream	It makes the scenario deterministic and testable.	No ingestion contract, schema evolution, producer validation, idempotency, or malformed-event quarantine.
Single-threaded replay	It removes concurrency from the assessment and makes causal ordering clear.	Real authorization races, locking, retry behavior, and partition handling are not solved.
Day-level dates	The supplied requirement uses Day 1 through Day 6.	Time zones, business calendars, cut-off times, daylight-saving issues, and timestamp ordering are deferred.
Hard-coded fee and interest constants	The numbers are part of the exercise and belong in the documented design.	Product configuration, effective-dated policy changes, approvals, and regulatory rate controls are absent.
Simple authorization states	Approved, declined, and settled are enough for the supplied events.	Expiry, cancellation, partial settlement, pending external outcomes, and dispute handling are deferred.
One account currency	Each account has one currency in the scenario.	FX rates, cross-currency settlement, rounding across currencies, and currency restrictions are not addressed.
No authentication/authorization	There are no API users or operators in this exercise.	No protection against unauthorized postings, administrative misuse, or segregation-of-duties violations.
No observability or deployment platform	The deliverable is a local runnable program.	No metrics, tracing, alerting, health checks, structured audit logs, or production deployment controls.

5. Production direction

If the core moved to production, I would preserve the domain concepts but add boundaries around them. A stateless ledger service would accept authenticated commands, validate and persist an immutable transaction plus its entries in one database transaction, and publish an integration event through an outbox. A projection worker would maintain fast balance and statement views, while reconciliation would periodically rebuild or compare balances from the source entries.

For storage, I would use a relational database such as PostgreSQL because the system needs atomic writes, foreign keys, unique constraints, and reliable transactions. I would use a transaction header plus one or more immutable ledger lines, with a double-entry design for real money movement. A unique idempotency key would protect command retries, and account-level concurrency control would protect authorization decisions.

I would treat the current in-memory engine as the business-rule core, not as the final production architecture. That keeps the exercise main strength the rules are easy to test while adding the durability, security, operational control, and scale mechanisms that production requires.